

 Projektpräsentation

pxsng

The AI Training Manager

Eine Web App zum Verwalten von Schulungen

Fynn Frontend Marco Backend Tim Datenbank



Drei Bereiche, drei Personen

Jeder von uns hat einen Bereich übernommen und stellt ihn kurz vor.

● TEIL 1

Frontend

Was man im Browser sieht. Startseite, Kurse, Suche, Profil.

Fynn . ca. 5 Minuten

● TEIL 2

Backend

Was im Hintergrund läuft. Holt Daten und schickt sie an die Webseite.

Marco . ca. 5 Minuten

● TEIL 3

Datenbank

Wo alle Daten gespeichert sind. Kurse, Teilnehmer, Rechnungen.

Tim . ca. 5 Minuten

Schulungen verwalten kostet zu viel Zeit

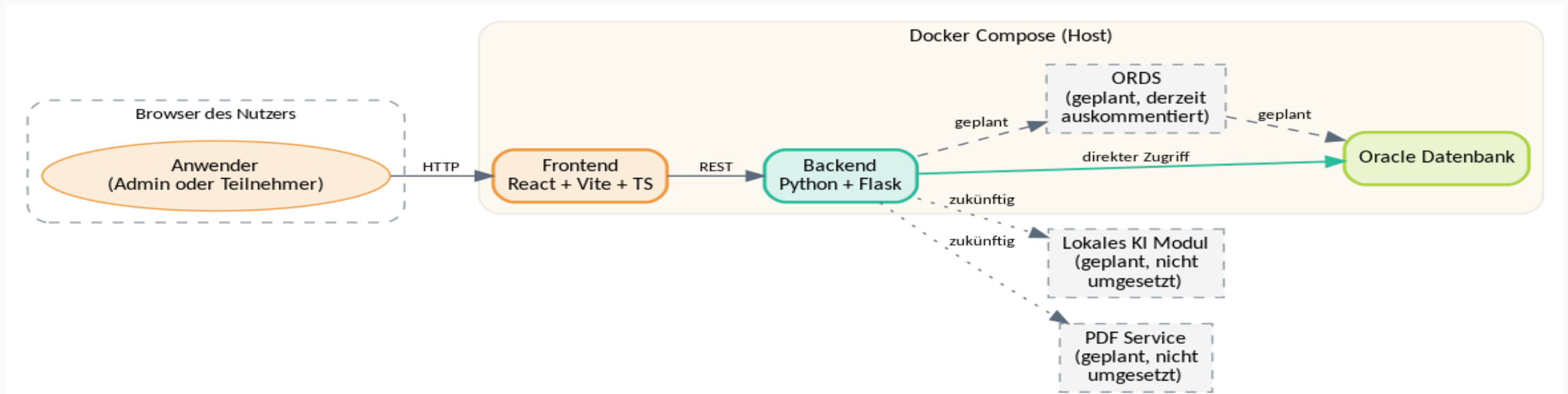
Heute

- Excel Listen, Word Vorlagen und E Mails
- jeder pflegt seine eigene Datei
- Fehler schleichen sich ein
- keiner sieht den Gesamtüberblick

Mit pxsng

- alle Daten an einem Ort
- Angebot und Rechnung als PDF
- KI schreibt Textbausteine automatisch
- eigene Webseite für Teilnehmer und Admins

So spielen die Teile zusammen



Frontend zeigt die Seite an, **Backend** liefert die Daten, **Datenbank** speichert sie.

Gestrichelte Teile sind noch nicht gebaut.

TEIL 1 . SAMMY

Frontend

Was die Nutzer im Browser sehen.

Was sehen die Nutzer



Startseite

- Zeigt eine klare Übersicht über verschiedene Kurse innerhalb definierter Kategorien.
- Kurse werden in einem Karousel bereitgestellt um dem Benutzer das Gefühl von "Endlosem" Content zu geben



Navigationsleiste

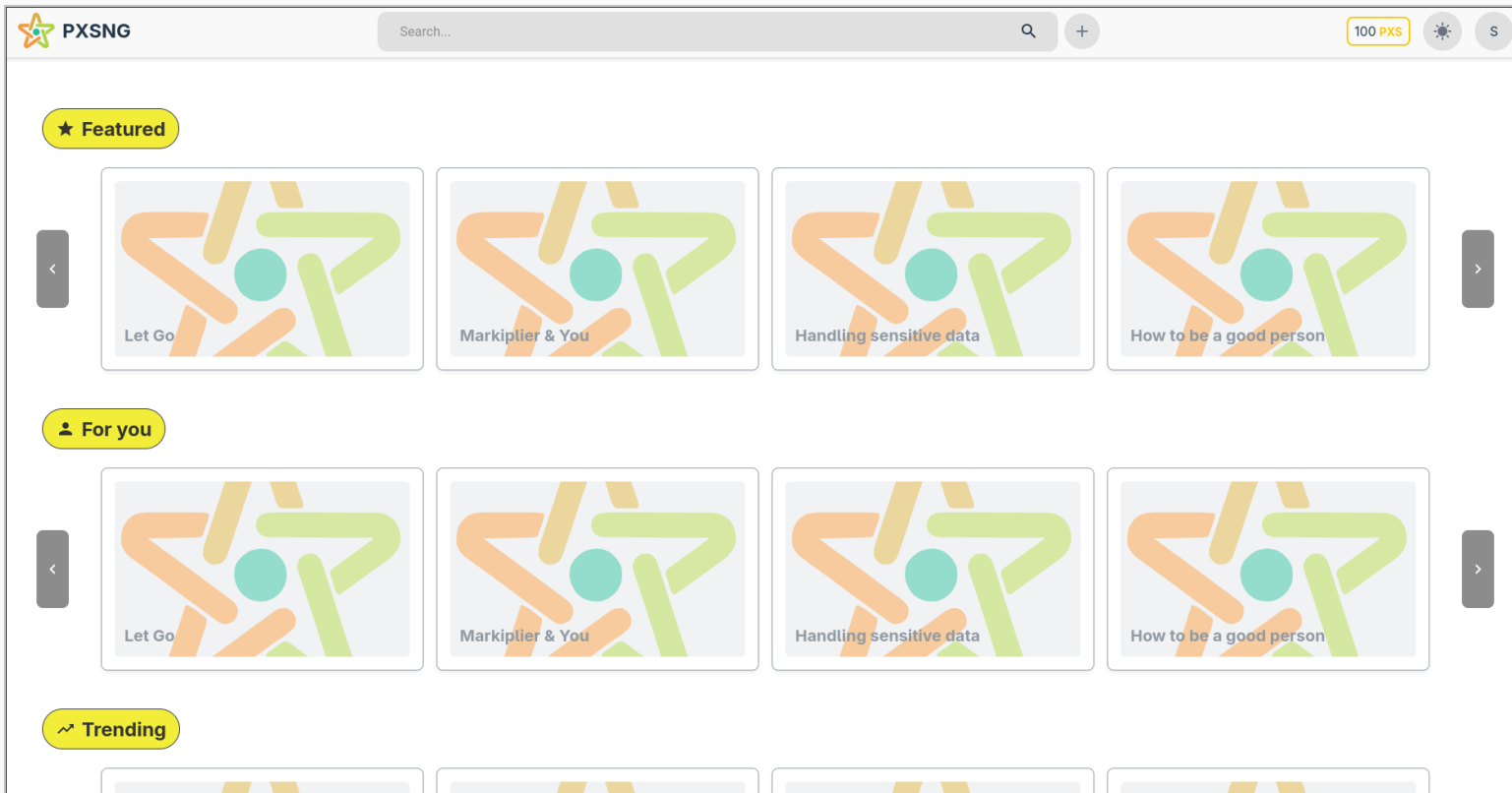
- Zeigt die wichtigsten Optionen und Informationen in einer geordneten Reihenfolge
- Website-Logo
- Zeigt den Benutzer und Währungsstand
- Klar definierte Suchleiste
- Theme-Toggle



Themes

Das Frontend kann auf Knopfdruck von einer hellen Anzeige in eine augenschonende dunklere Anzeige umgewandelt werden.

Was sehen die Nutzer



Startseite

- Zeigt eine klare Übersicht über verschiedene Kurse innerhalb definierter Kategorien.
- Kurse werden in einem Karousel bereitgestellt um dem Benutzer das Gefühl von "Endlosem" Content zu geben

Was sehen die Nutzer



Startseite

- Zeigt eine klare Übersicht über verschiedene Kurse innerhalb definierter Kategorien.
- Kurse werden in einem Karousel bereitgestellt um dem Benutzer das Gefühl von "Endlosem" Content zu geben



Navigationsleiste

- Zeigt die wichtigsten Optionen und Informationen in einer geordneten Reihenfolge
- Website-Logo
- Zeigt den Benutzer und Währungsstand
- Klar definierte Suchleiste
- Theme-Toggle



Themes

Das Frontend kann auf Knopfdruck von einer hellen Anzeige in eine augenschonende dunklere Anzeige umgewandelt werden.

Was sehen die Nutzer



Navigationsleiste

- Zeigt die wichtigsten Optionen und Informationen in einer geordneten Reihenfolge
- Website-Logo
- Zeigt den Benutzer und Währungsstand
- Klar definierte Suchleiste
- Theme-Toggle



Was sehen die Nutzer



Startseite

- Zeigt eine klare Übersicht über verschiedene Kurse innerhalb definierter Kategorien.
- Kurse werden in einem Karousel bereitgestellt um dem Benutzer das Gefühl von "Endlosem" Content zu geben



Navigationsleiste

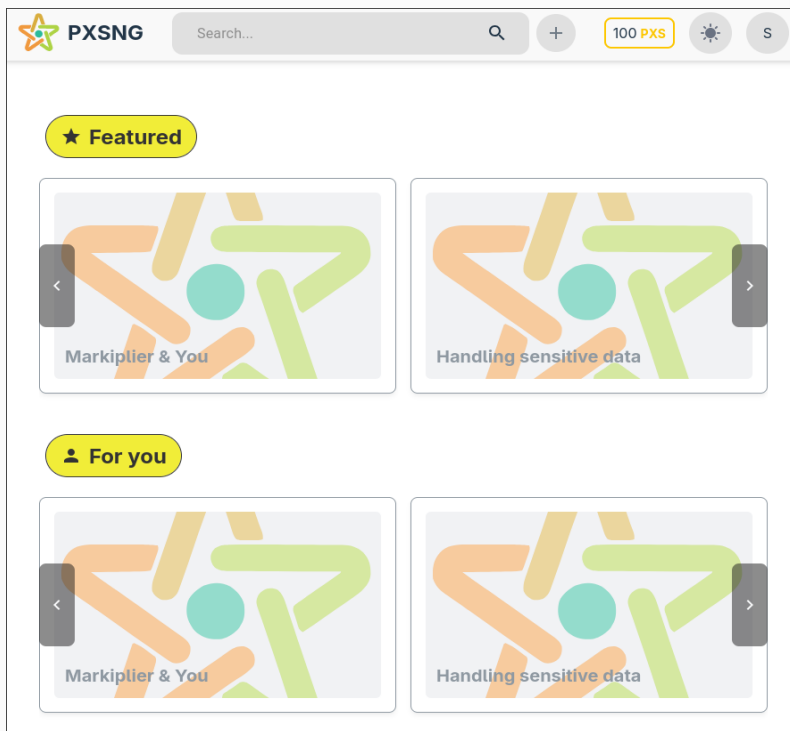
- Zeigt die wichtigsten Optionen und Informationen in einer geordneten Reihenfolge
- Website-Logo
- Zeigt den Benutzer und Währungsstand
- Klar definierte Suchleiste
- Theme-Toggle



Themes

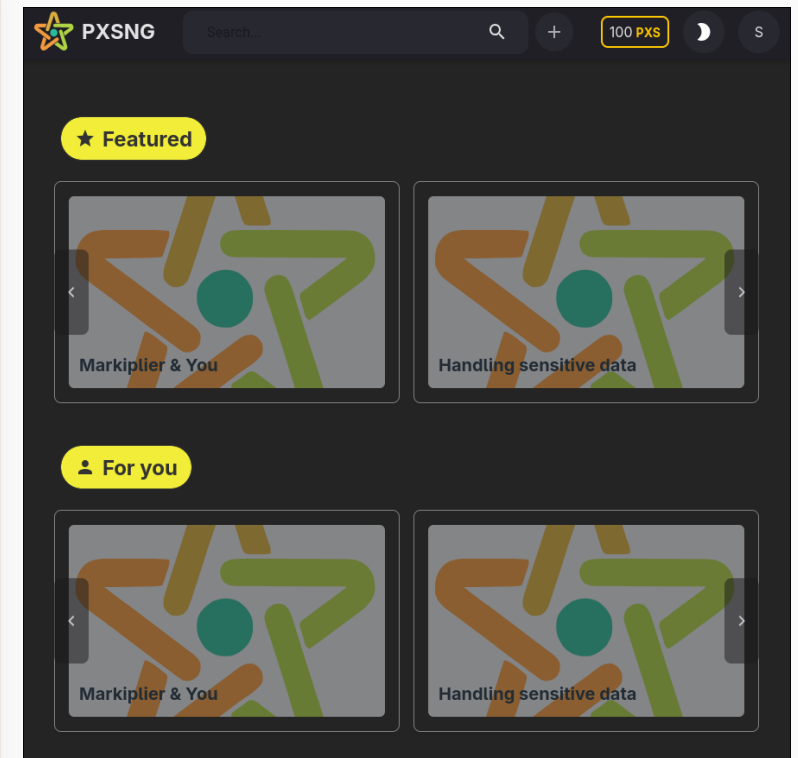
Das Frontend kann auf Knopfdruck von einer hellen Anzeige in eine augenschonende dunklere Anzeige umgewandelt werden.

Was sehen die Nutzer



Themes

Das Frontend kann auf Knopfdruck von einer hellen Anzeige in eine augenschonende dunklere Anzeige umgewandelt werden.



Der Tech-Stack

● React

Ein UI-Framework bekannt für die einfache Handhabung

● TypeScript

Eine abwandlung von JavaScript die mögliche Fehler früh erkennt und aufzeigt

● Vite

Schnelles und Effizientes Kompilieren, sowie ein schneller Development Server

● Material UI

Funktionale und Vorgefertigte UI-Elemente um Entwicklung zu verschnellern.

● TailwindCSS

UI-Library um das Anfertigen von Styles schneller zu gestalten

Do's und Dones

Schon fertig

- ✓ Startseite mit Kursen aus dem Backend
- ✓ Top Leiste mit Suche
- ✓ Theming: hell und dunkel
- ✓ Verbindung zum Backend
- ✓ Responsive Design

Kommt noch

- Login und Registrierung
- Kurs buchen mit Warenkorb
- Eigene Buchungen und Zertifikate
- Adminbereich für die Verwaltung
- Kursteilnahme

TEIL 2 . MARCO

Backend

Was im Hintergrund passiert.

Was macht das Backend

 SCHRITT 1

Frage empfangen

Die Website (Frontend) sendet eine HTTP-Anfrage an das Backend – beispielsweise, um einen bestimmten Kurs über `/courses/1` oder einen Benutzer über `/users/UUID` abzufragen

 SCHRITT 2

Daten holen

Das Python-Backend kommuniziert mit dem Oracle 23ai-Datenbankcontainer, um die angeforderten Datensätze abzurufen.

 SCHRITT 3

Antwort schicken

Das Backend verpackt die abgerufene Datenbankzeile und sendet sie als saubere JSON-Antwort an das Frontend zurück.

Gebaut mit Python und einer Bibliothek namens Flask. Schlank und gut zu lesen.

Was macht das Backend

Warum Flask-RESTX die ideale Wahl ist

Typisierung & Validierung

- Anpassungen an exakten JSON-Schema auf Python-Ebene
- Fehler werden sofort auf Router-Ebene erkannt

Automatisierte Dokumentation

- Einsatz von SwaggerAPI und Namespaces
- Es dokumentiert sich selbst

 SCHRITT 1

Frage empfangen

Die Website (Frontend) sendet eine HTTP-Anfrage an das Backend – beispielsweise, um einen bestimmten Kurs über `/courses/1` oder einen Benutzer über `/users/UUID` abzufragen

Gebaut mit Python und einer Bibliothek namens Flask. Schlank und gut zu lesen.

Was macht das Backend

 SCHRITT 1

Frage empfangen

Die Website (Frontend) sendet eine HTTP-Anfrage an das Backend – beispielsweise, um einen bestimmten Kurs über `/courses/1` oder einen Benutzer über `/users/UUID` abzufragen

 SCHRITT 2

Daten holen

Das Python-Backend kommuniziert mit dem Datenbankcontainer, um die angeforderten Datensätze abzurufen.

 SCHRITT 3

Antwort schicken

Das Backend verpackt die abgerufene Datenbankzeile und sendet sie als saubere JSON-Antwort an das Frontend zurück.

Gebaut mit Python und einer Bibliothek namens Flask. Schlank und gut zu lesen.

Was macht das Backend

● SCHRITT 2

Daten holen

Das Python-Backend kommuniziert mit dem Datenbankcontainer, um die angeforderten Datensätze abzurufen.

Schwierigkeiten

- Das Phänomen der Endlosschleife
 - PL/SQL
- Die Ursache
 - IN- und OUT Parametern in oracledb

```
cursor.callproc(  
    "course_api_pkg.get_course_by_id",  
    [id, p_course_cursor]  
)  
  
cursor.execute(  
    """  
    SELECT * FROM "COURSE"  
    WHERE id = :1  
    """, [id],  
)
```

Gebaut mit Python und einer Bibliothek namens Flask. Schlank und gut zu lesen.

Was macht das Backend

 SCHRITT 1

Frage empfangen

Die Website (Frontend) sendet eine HTTP-Anfrage an das Backend – beispielsweise, um einen bestimmten Kurs über `/courses/1` oder einen Benutzer über `/users/UUID` abzufragen

 SCHRITT 2

Daten holen

Das Python-Backend kommuniziert mit dem Datenbankcontainer, um die angeforderten Datensätze abzurufen.

 SCHRITT 3

Antwort schicken

Das Backend verpackt die abgerufene Datenbankzeile und sendet sie als saubere JSON-Antwort an das Frontend zurück.

Gebaut mit Python und einer Bibliothek namens Flask. Schlank und gut zu lesen.

Was macht das Backend

● SCHRITT 3

Antwort schicken

Das Backend verpackt die abgerufene Datenbankzeile und sendet sie als saubere JSON-Antwort an das Frontend zurück.

Datenserialisierung

```
user_model = api.model(  
    "User",  
    {  
        "id": fields.String(),  
        "firstname": fields.String(),  
        ...  
    },  
)
```

FlaskX Data Marshalling

```
@api.route("/<string:id>")  
class UserController(Resource):  
    @api.marshal_with(user_model)  
    def get(self, id):  
        ...
```

Gebaut mit Python und einer Bibliothek namens Flask. Schlank und gut zu lesen.

Was macht das Backend

● SCHRITT 3

Das Backend verpackt die abgerufene Datenbankzeile und sendet sie als saubere JSON-Antwort an das Frontend zurück.

Antwort schicken

Code	Description
200	Success
	Example Value Model <pre>{ "id": "string", "firstname": "string", "lastname": "string", "email": "string", "telephone": "string", "password_hash": "string", "address_id": "string" }</pre>

▼ definitions:	
▶ Course:	{ type: "object", properties: {...} }
▼ User:	
▼ properties:	
▼ id:	type: "string"
▼ firstname:	type: "string"
▼ lastname:	type: "string"
▼ email:	type: "string"
▼ telephone:	type: "string"
▼ password_hash:	

Gebaut mit Python und einer Bibliothek namens Flask. Schlank und gut zu lesen.

Eine Sache lief nicht wie geplant

Geplant war

Wir wollten einen Vermittler zwischen Backend und Datenbank benutzen, der die Arbeit vereinfacht. Das Werkzeug heißt ORDS und kommt von Oracle.

Was geworden ist

Das Setup hat einfach nicht stabil funktioniert. Bei jedem Versuch hatten wir neue Probleme. Im Lauf des Projekts haben wir es daher sein gelassen und sprechen direkt mit der Datenbank.

Wir lassen das Thema bewusst offen für die nächste Klasse.

Do's und Dones

Schon fertig

- ✓ API-Endpunkte für Kurse und Benutzer
- ✓ Datenbankverbindung mit Pooling als Ressourcenverwaltung
- ✓ CORS
- ✓ Datenbank SQL-Abfragen

Kommt noch

- API-Endpunkte für andere Tabellen aus der Datenbank (Warenkorb, Tokens, Rechnungen und Firmen)
- Erzeugungen von PDF Zertifikate
- KI Integration

TEIL 3 . TIM

Datenbank

Wo alle Daten gespeichert sind.

Was wird gespeichert?

Ein umfangreiches, extensives Custom-Schema, das direkt aus unserem ER-Diagramm für alle künftigen Features generiert wurde.

STRUKTUR

16 Tabellen & 22 Beziehungen

Umfasst alle logischen Bereiche wie Stammdaten, Personen, Kurse und Rechnungsdaten sauber normalisiert.

PROJEKT-ZIEL

Zukunftssicher aufgebaut

Das Schema ist direkt so ausgelegt, dass wir alle aktuellen und künftigen Use-Cases direkt abbilden können.

Vorteil: Datenintegrität durch ein professionell modelliertes ER-Schema.

Die Architektur wurde angepasst

Eigentlich sollte ORDS die REST-Brücke schlagen – wir haben uns im Projektverlauf jedoch für einen direkteren Weg entschieden.

GEPLANT

ORDS REST-API (Bypass)

Die Kommunikation über Oracle REST Data Services scheiterte an anhaltenden Setup- und Stabilitätsproblemen.

UMSETZUNG

Direkte SQL-Anbindung

Das Flask-Backend spricht nun nativ über Python-Treiber direkt mit der Oracle-Datenbank für maximale Performance.

Entscheidung: Mehr Stabilität und Kontrolle ohne den fehleranfälligen ORDS-Vermittler.

Die Datenbank macht selber mit

Wir haben einen Teil der Logik nicht ins Backend gepackt, sondern direkt in die Datenbank. Damit kann sie kleine Aufgaben selbst erledigen.

BEISPIEL

User anlegen

Beim Anlegen eines Users wird automatisch ein passender Warenkorb mit erzeugt. Das Backend muss daran nicht denken.

BEISPIEL

Datum automatisch

Wenn ein Datensatz geändert wird, schreibt die Datenbank automatisch das aktuelle Datum mit.

Vorteil: ein einheitliches Verhalten, egal wer aufruft.

Hürden beim Docker-Setup

Der lokale Datenbank-Container läuft zwar stabil, aber das automatische Deployen der DB-Objekte gestaltete sich als extrem zeitintensiv.

SETUP

Automatisches Deployment

Schema, User, Packages und Procedures vollautomatisch beim Container-Startup einzuspielen, war eine der größten Herausforderungen.

PROBLEMATIK

Inkonsistente Ergebnisse

Aufgrund von Timing-Issues beim Starten der Oracle-Instanz im Docker-Container läuft das Deployment bis heute nicht deterministisch.

Ergebnis: Schema-Elemente mussten für Tests teilweise manuell eingepflegt werden.

Inkompatibilitäten & Testdaten

Sowohl Betriebssystem-Unterschiede im Team als auch die fehlende Generierung von Beispieldaten erschwerten die Testphasen.

BETRIEBSSYSTEM

Windows vs. Linux Setup

Volume-Mounts unter Windows Docker Desktop führten oft zu Berechtigungsfehlern in den Oracle-Ordnerstrukturen.

STATUS

Testdaten fehlen

Aktuell steht nur die Struktur bereit. Ohne konsistente Testdaten in der DB konnten noch keine realen Testläufe abgeschlossen werden.

Ziel nächste Klasse: Auto-Deployment stabilisieren und synthetische Testdaten generieren.

So macht die nächste Klasse weiter

1

Beispieldaten austauschen

Im Backend die letzten Beispieldaten gegen echte Datenbankaufrufe ersetzen.

2

PDF Service bauen

Modul, das Angebote, Rechnungen und Zertifikate als PDF erzeugt.

3

KI anbinden

Lokale KI für Textbausteine, zum Beispiel über Ollama.

4

Adminbereich

Eigener Bereich mit Formularen und Übersichten für die Verwaltung.

5

Login einbauen

Anmeldung mit Passwort, Schutz der Seiten.

6

Über ORDS entscheiden

Noch einmal versuchen oder endgültig entfernen.

F R A G E N

Fragen?

Wir freuen uns auf eure Anmerkungen.